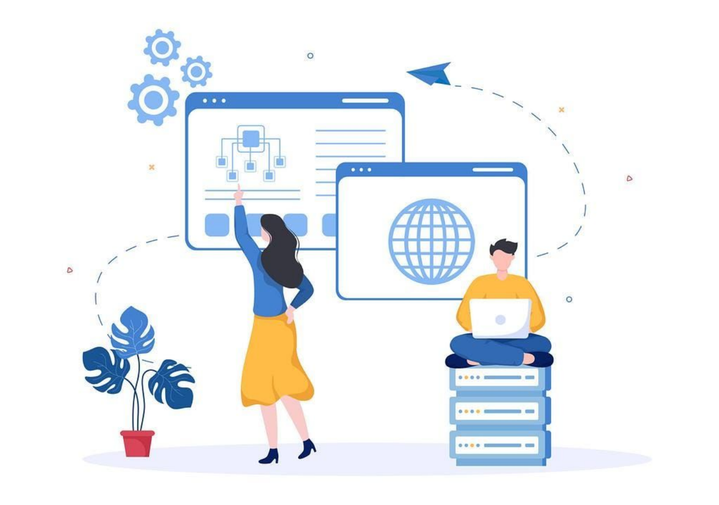
Google Developer Group
Universitas Sriwijaya

Learning 5

State Management

State and local data management using useState



Static UI vs Dynamic UI

- UI does not change after rendering

Static UI

```
<h1>Hello World</h1>
```

Dynamic UI

```
<h1>{count}</h1>
```

What is State

Data that can change inside a component

- Stored inside a component
- Can change over time
- When state changes, React re-renders the component

Common use cases: Counter, Toggle UI, Show/Hide Content, User Interaction

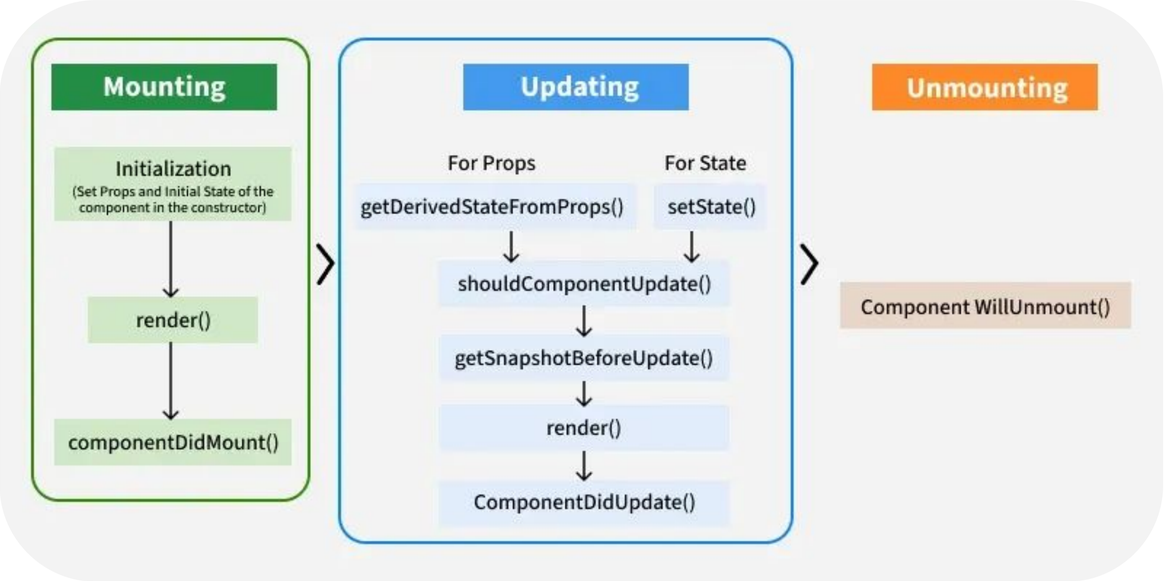
React Hooks

React provides Hooks to manage component logic.

Hooks allow functional components to use:

- State
- Lifecycle features

The most basic Hook: *useState()*



useState Hook

useState is used to create and manage state.

Explanation:

- *count* → current state value
- *setCount* → function to update state
- 0 → initial state value

```
const [count, setCount] = useState(0)
```

Simple Counter Example

```
const [count, setCount] = useState(0)

<button onClick={() => setCount(count + 1)}>
  Add
</button>

<p>{count}</p>
```

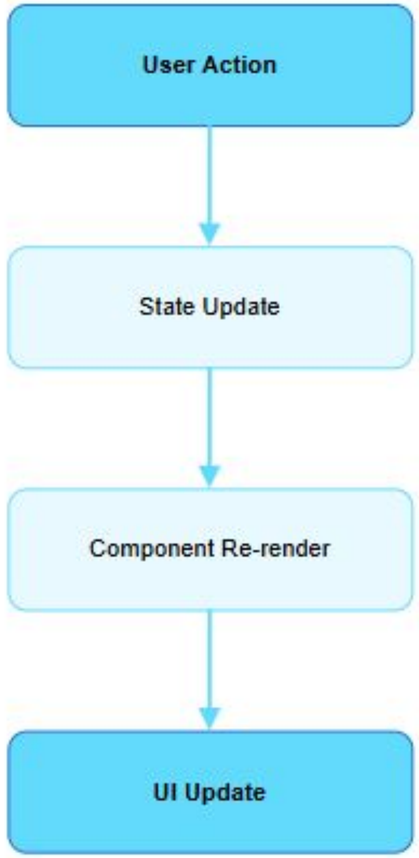
Flow:

1. User clicks the button
2. *setCount* updates the state
3. React re-renders the component
4. UI updates automatically

State Triggers Re-render

When state changes:

1. React runs the component again
2. React recalculates the UI
3. The UI updates on the screen



Conditional Rendering with State

State is often used together with conditional rendering.

```
const [isOpen, setIsOpen] = useState(false)

<button onClick={() => setIsOpen(!isOpen)}>
  Toggle
</button>

{isOpen && <p>Content Visible</p>}
```

This allows UI to appear or disappear based on state.

Basic Best Practices

- Use state only for data that changes
- Never update state directly
- Always use the setter function
- State is local to the component

Google Developer Group
Universitas Sriwijaya

Thank You !